

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 Milestone 1

Student: Mariano R. (mr822)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 7/7/2025 2:33:18 PM

Updated: 7/7/2025 4:21:58 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/grading/mr822>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/view/mr822>

Instructions

- Overview Link: <https://youtu.be/9dZPFwi76ak>

1. Refer to Milestone1 of any of these docs:
 2. [Rock Paper Scissors](#)
 3. [Basic Battleship](#)
 4. [Hangman / Word guess](#)
 5. [Trivia](#)
 6. [Go Fish](#)
 7. [Pictionary / Drawing](#)
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy Part5 and rename the copy as Project (this new folder should be in the root of your repo)
6. Organize the files into their respective packages Client, Common, Server, Exceptions
 1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
 1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to main
10. Upload the same PDF to Canvas
11. Sync Local

1. git checkout main
2. git pull origin main

Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections

```
mramos2001@DESKTOP-US058A0:~/IT114$ java -cp . Project.Server.Server
07/07/2025 14:30:51 [Project.Server.Server] (INFO):
> Server Starting
07/07/2025 14:30:51 [Project.Server.Server] (INFO):
> Server: Listening on port 3000
07/07/2025 14:30:51 [Project.Server.Room] (INFO):
> Room[lobby]: Created
07/07/2025 14:30:51 [Project.Server.Server] (INFO):
> Server: Created new Room lobby
07/07/2025 14:30:51 [Project.Server.Server] (INFO):
> Server: Waiting for next client
```

SS Terminal

```
// USER: mramos2001
// DATE: 04/24/25 07
// THIS SNIPPET SHOWS THE SERVER LISTENING FOR INCOMING CONNECTIONS

private void start(int port) {
    this.port = port;
    // server listening
    info("Listening on port " + this.port);
    // simulated client connections loop
    try {
        ServerSocket serverSocket = new ServerSocket(port);
        while (true) {
            info("Looping for next client");
            Socket incomingClient = serverSocket.accept(); // blocking call, waits for a client connection
            info("New connection");
            // send socket to a new thread, pass a call to Socket IncomingClient
            // create a new thread
            ServerThread serverThread = new ServerThread(incomingClient, this.getServerThreadList());
            // send the thread (initially an internal party manager, the thread will be
            // don't have the thread start itself
            // we'll wait for it
            // Note: we don't yet add the ServerThread reference to our currentClients map
        }
    } catch (IOException e) {
        logger.warn("Server failed to start", e);
    }
    info("Closing server socket");
}
```

SS Code

Saved: 7/7/2025 2:38:12 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

The server calls `serverSocket.accept()`, which blocks until a client connects. It then creates a `ServerThread` to handle communication with that client in parallel.



Saved: 7/7/2025 2:38:12 PM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

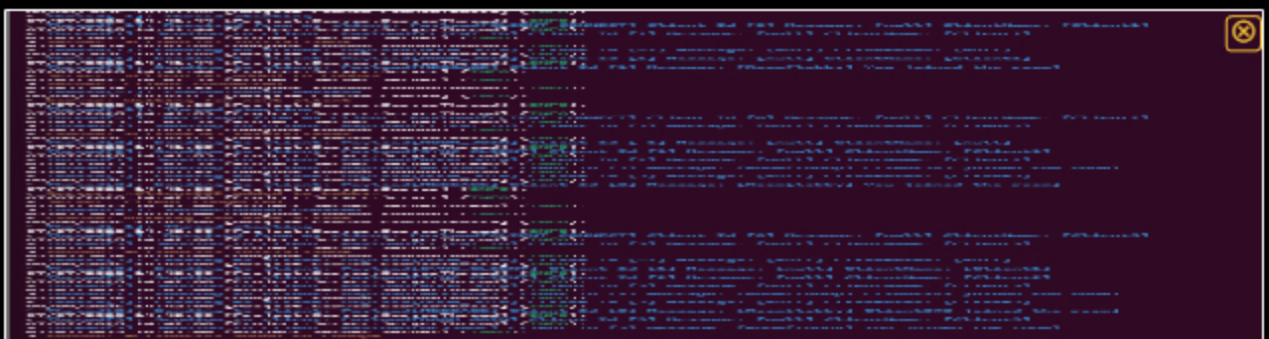
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections



SS Server

```
try (ServerSocket serverSocket = new ServerSocket(port)) {
    createRoom(Room.LOBBY); // create the first room (lobby)
    // UCID: wramos2001
    // Date: 2025-07-07
    // This snippet shows how the server handles multiple clients in separate threads.

    while (isRunning) {
        info("Waiting for next client");
        Socket incomingClient = serverSocket.accept(); // blocking action, waits for a client connection
        info("Client connected");
        // wrap socket in a ServerThread, pass a callback to notify the Server when
        // they're initialized
        ServerThread serverThread = new ServerThread(incomingClient, this::onServerThreadInitialized);
        // start the thread (typically an external entity manages the lifecycle and we
        // don't have the thread start itself)
        serverThread.start();
        // Note: We don't yet add the ServerThread reference to our connectedClients map
    }
}
```

SS Code1

```
// ID: 111, 111, 111, 111
// Date: 2025-07-07
// This snippet shows each client is handled in a separate thread.

@Override
public void run() {
    info("Thread starting");
    try {
        ObjectOutputStream out = new ObjectOutputStream(client.getOutputStream());
        ObjectInputStream in = new ObjectInputStream(client.getInputStream());
        this.out = out;
        isRunning = true;
        new java.util.Timer().schedule(new java.util.TimerTask() {
            @Override
            public void run() {
                if (getClientName() == null || getClientName().isEmpty()) {
                    info("Client name not received. Disconnecting");
                    disconnect();
                }
            }
        }, 10000);
        Payload = fromClient;
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

SS Code2



Saved: 7/7/2025 3:16:14 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients

Your Response:

The server accepts each client connection and creates a new ServerThread for it. Each thread runs independently, allowing multiple clients to be handled in parallel without blocking the main server loop.



Saved: 7/7/2025 3:16:14 PM

Section #3: (2 pts.) Feature: Server Will Implement The Concept Of Rooms (With The Default Being "l obby")

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

```

// UCID: mramos2001
// Date: 2025-07-07
// This snippet shows the server handling room creation and joining.

/**
 * Attempts to create a new Room and add it to the tracked rooms collection
 *
 * @param name Unique name of the room
 * @return true if it was created and false if it wasn't
 * @throws DuplicateRoomException
 */
protected void createRoom(String name) throws DuplicateRoomException {
    final String nameCheck = name.toLowerCase();
    if (rooms.containsKey(nameCheck)) {
        throw new DuplicateRoomException(String.format("Room %s already exists", name));
    }
    Room room = new Room(name);
    rooms.put(nameCheck, room);
    info(String.format("Created new Room %s", name));
}

```

SS Code 1

```

// UC ID: mramos2001
// Date: 2025-07-07
// This snippet shows the server handling room creation and joining.

/**
 * Attempts to move a client (ServerThread) between rooms
 *
 * @param name The target room to join
 * @param client the client moving
 * @throws RoomNotFoundException
 */
protected void joinRoom(String name, ServerThread client) throws RoomNotFoundException {
    final String nameCheck = name.toLowerCase();
    if (!rooms.containsKey(nameCheck)) {
        throw new RoomNotFoundException(String.format("Room %s wasn't found", name));
    }
    Room currentRoom = client.getCurrentRoom();
    if (currentRoom != null) {
        info("Removing client from previous Room " + currentRoom.getName());
        currentRoom.removeClient(client);
    }
    Room next = rooms.get(nameCheck);
    next.addClient(client);
}

```

SS Code 2

```

// UCID: mramos2001
// Date: 2025-07-07
// This snippet shows the server handling room creation and joining.

protected void removeRoom(Room room) {
    rooms.remove(room.getName().toLowerCase());
    info(String.format("Removed room %s", room.getName()));
}

```

SS Code 3

```

// UCID: mramos2001
// Date: 2025-07-07
// This snippet shows the server handling room creation and joining.

@Override
public void close() {
    // attempt to gracefully close and migrate clients
    if (!isEventsInRoom.isEmpty()) {
        info("Room is still being cleaned, migrating the lobby");
        info(String.format("Migrating %d clients to the lobby", clientsInRoom.values().size()));
        try {
            Server.INSTANCE.joinRoom(Room.LOBBY, client);
        } catch (RoomNotFoundException e) {
            e.printStackTrace();
            // lobby, this shouldn't happen though
        }
        return true;
    }
    Server.INSTANCE.removeRoom(this);
    isRunning = false;
    clientsInRoom.clear();
    info(String.format("closed"));
}

```

SS Code 4

```

// UC ID: mramos2001
// Date: 2025-07-07
// This snippet shows the server handling room creation and joining.

public void handleCreateRoom(ServerThread sender, String roomName) {
    try {
        Server.INSTANCE.createRoom(roomName);
        Server.INSTANCE.joinRoom(roomName, sender);
    } catch (RoomNotFoundException e) {
        info("Room wasn't found (this shouldn't happen)");
        e.printStackTrace();
    } catch (DuplicateRoomException e) {
        sender.sendMessage(Constants.DEFAULT_CHANNEL_ID, String.format("Room %s already exists", roomName));
    }
}

public void handleJoinRoom(ServerThread sender, String roomName) {
    try {

```

```
Server.INSTANCE.joinRoom(roomName, sender);
} catch (RoomNotFoundException e) {
    sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format("Room %s doesn't exist", roomName));
}
```

SS Code 5

```
Room room = null;
try {
    room = Server.INSTANCE.getRoom(roomName);
} catch (RoomNotFoundException e) {
    // Room not found, create a new room
    room = new Room(roomName);
    Server.INSTANCE.addRoom(room);
}

// Add the sender to the room
room.addClient(sender);

// Notify the sender that they have joined the room
sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format("You have joined the room %s", roomName));

// Notify other clients in the room that a new client has joined
for (Client client : room.getClients()) {
    client.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format("New client joined the room %s", roomName));
}
```

SS Terminal

 Saved: 7/7/2025 3:30:36 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

The server maintains a `ConcurrentHashMap` of rooms. When a client requests to create a room, `createRoom()` checks if it already exists and adds it to the map. When joining or leaving, `joinRoom()` and `Room.handleJoinRoom()/handleLeaveRoom()` move the client between rooms and notify other clients. If a room becomes empty (except the lobby), `Room.close()` is called to remove it from the server and free resources.

 Saved: 7/7/2025 3:30:36 PM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the `/name` and `/connect` commands for each of 3 clients (best

- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected

SS Terminal

SS Code 1SS Code 2

Progress: 100%

Your Response:

When the user types /name, the client saves the chosen name locally. Typing /connect host:port creates a socket connection to the server and starts listening. The client then sends its name to the server. The server responds with a client ID, which the client stores to confirm the connection is successful.

 Saved: 7/7/2025 3:23:11 PM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

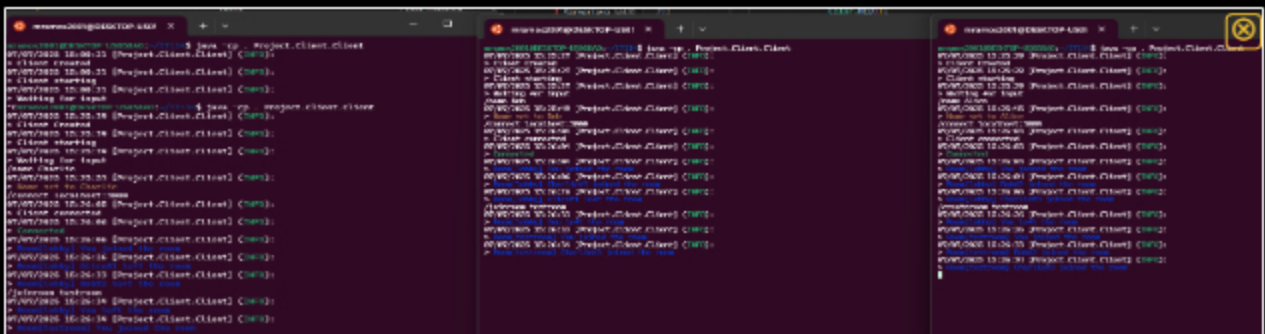
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining



The image shows three terminal windows side-by-side. The left window shows the output of the /createroom command, including a success message and a room ID. The middle window shows the output of the /joinroom command, including a success message and a room ID. The right window shows the output of the /joinroom command, including a success message and a room ID.

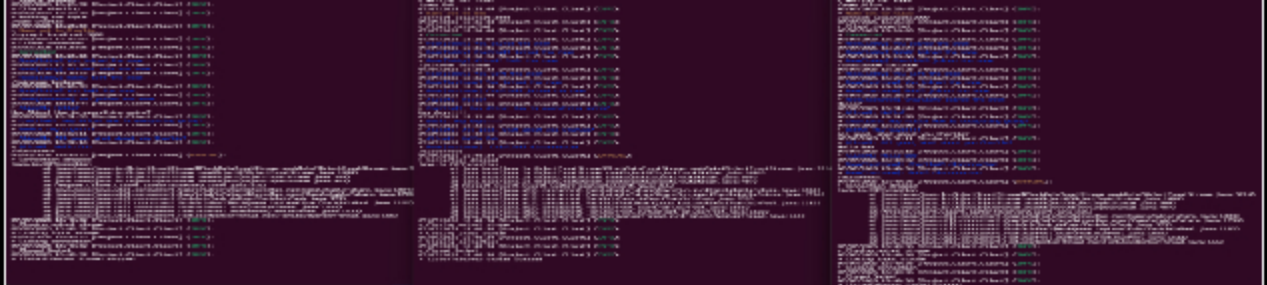
SS Terminal

```
wasCommand = true;
// UCTD: Miramax2001
// Date: 2025-07-07
// This snippet shows how the client processes /createroom and /joinroom commands.

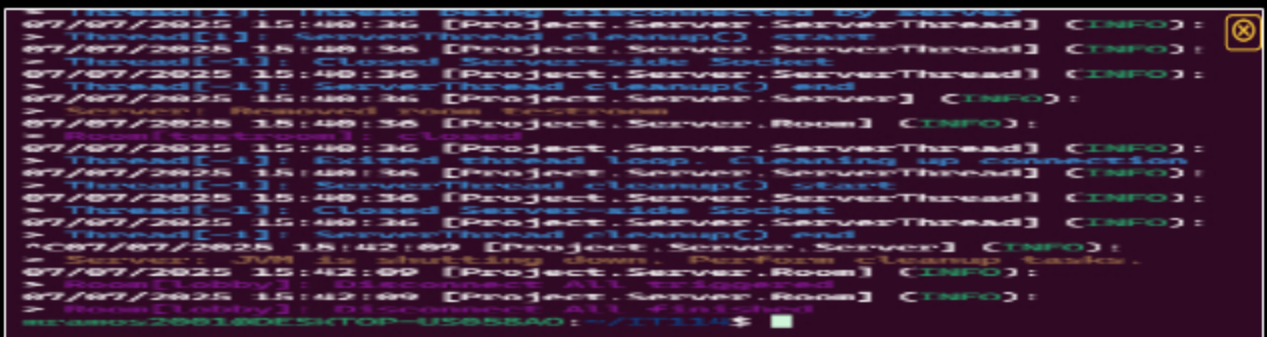
} else if (text.startsWith(Command.CREATE_ROOM.command)) {
    text = text.replace(Command.CREATE_ROOM.command, "").trim();
    if (text == null || text.length() == 0) {
        LoggerUtil.INSTANCE.warning(TextFX.colorize("This command requires a room name as an argument", Color.RED));
        return true;
    }
    sendRoomAction(text, RoomAction.CREATE);
    wasCommand = true;
} else if (text.startsWith(Command.JOIN_ROOM.command)) {
    text = text.replace(Command.JOIN_ROOM.command, "").trim();
    if (text == null || text.length() == 0) {
        LoggerUtil.INSTANCE.warning(TextFX.colorize("This command requires a room name as an argument", Color.RED));
        return true;
    }
    sendRoomAction(text, RoomAction.JOIN);
    wasCommand = true;
}
```

SS Code

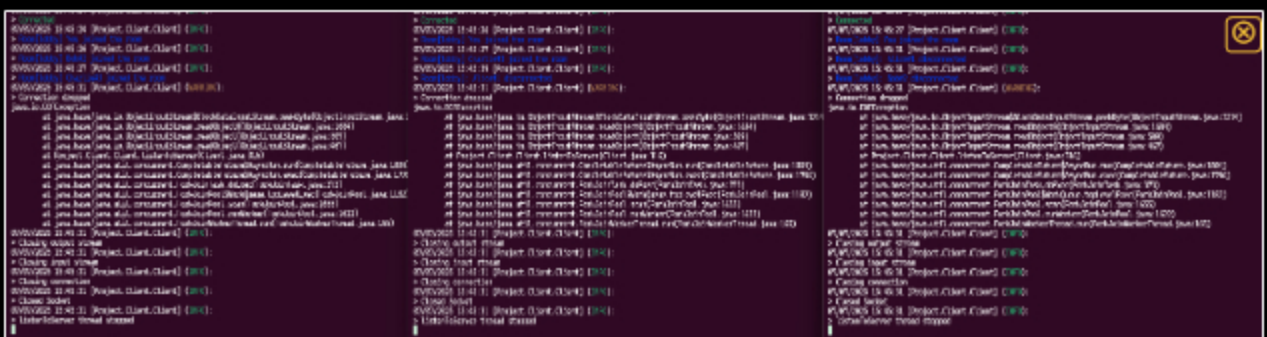
- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occurring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process



Client Disconnection



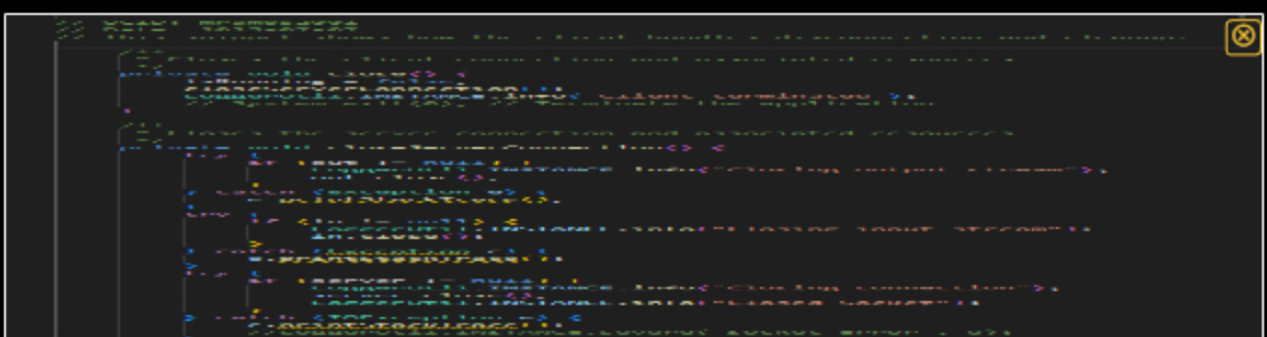
Server ended



Server ended client perspective



Client Reconnection



Tree



Saved: 7/7/2025 4:21:58 PM

☰ Task #2 (0.25 pts.) - Github Details

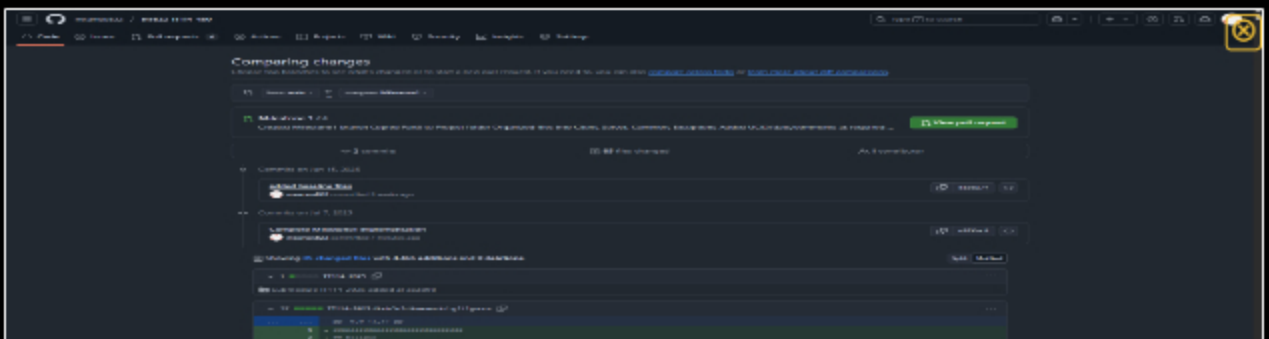
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



Github



Saved: 7/7/2025 4:18:10 PM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/mramos822/mr822-IT114-150/>



URL

<https://github.com/mramos822/n>



Saved: 7/7/2025 4:18:10 PM

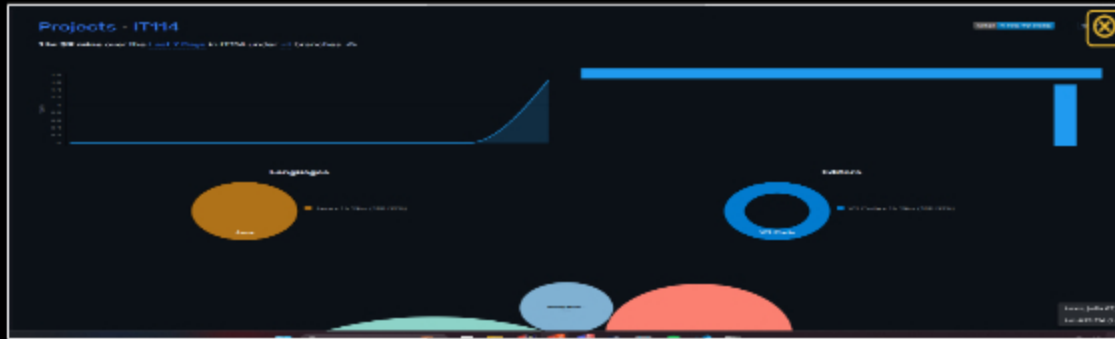
📁 Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard

- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- **Note:** The duration isn't relevant for the grade and the visual graphs aren't necessary



Waka1

Files		Branches	
51 mins	Client/Client.java	1 hr 32 mins	Milestone1
19 mins	Server/Server.java	7 mins	main
18 mins	Server/BaseServerThread.java		
7 mins	Server/Room.java		
1 min	Server/ServerThread.java		

Waka2



Saved: 7/7/2025 4:20:04 PM

≡ Task #4 (0.25 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to build a multi-threaded server in Java that can accept and manage connections from multiple clients at the same time. I also gained experience organizing a project into clear packages like Client, Server, Common, and Exceptions, which makes the code easier to maintain. Additionally, I practiced using Git and GitHub for version control, creating commits, branches, and pull requests to track my progress.



Saved: 7/7/2025 4:20:33 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was setting up the initial project structure and creating the basic Java classes. Organizing the files into packages like Client, Server, and Common was straightforward once I understood what each package was for. Running the server and client from the command line and seeing them connect was also pretty simple after the initial setup.



Saved: 7/7/2025 4:20:53 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was understanding and handling all the different message types and commands between the client and server. Making sure that rooms worked correctly when clients joined, left, or disconnected required a lot of careful testing. Debugging the disconnection issues and making sure everything cleaned up gracefully without crashing the server took the most time.



Saved: 7/7/2025 4:21:45 PM